

SỬ DỤNG IP CORES

Thực hành tìm hiểu phương pháp thiết kế onchip Memory trên FPGA sử dụng LPM IP cores và phương pháp sử dụng offchip Memory

Mục đích:

Trong những hệ thống máy tính, để lưu trữ chương trình và lưu trữ dữ liệu nó luôn cần một bộ nhớ với dung lượng đủ để lưu trữ. Nếu một hệ thống được thực thi trên FPGA, thì nó có thể sử dụng bộ nhớ có sẵn trên FPGA (onchip memory) do nhà sản xuất hỗ trợ sẵn hoặc ta cũng có thể thiết kế bộ nhớ ngay trên FPGA (onchip memory). Tuy nhiên, nếu dung lượng bộ nhớ trên FPGA không đủ thì một bộ nhớ bên ngoài FPGA (offchip memory) sẽ cần được sử dụng. Do đó, trong phần thực hành này ta sẽ tìm hiểu ba vấn đề sau:

- ❖ Tìm hiểu cách thức hiện thực một memory trên FPGA (onchip memory)
- ❖ Tìm hiểu cách thức sử dụng một offchip memory (SRAM chip trên kit DE2)
- ❖ Chạy mô phỏng trên ModelSim

1.1.1.1 Phần 1

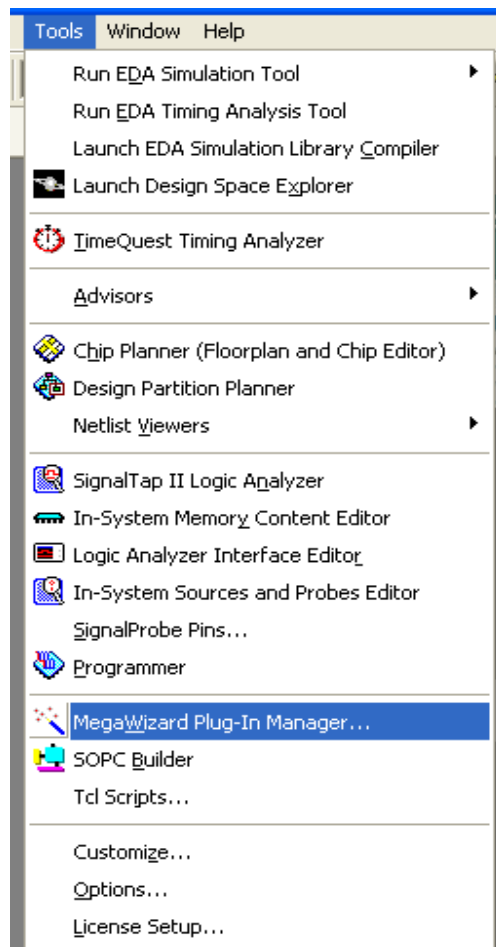
Altera cung cấp một thư viện có sẵn LPM chứa các thiết kế cơ bản như là adders, registers, counters và memories (Quartus II Library of Parameterized Modules). Ta có thể sử dụng các mô tả thiết kế này để thiết kế trên FPGA. Để thiết kế một bộ nhớ SRAM trên FPGA ta có thể dùng module `altsyncram` LPM do Altera cung cấp. Phần này ta sẽ thiết kế một SRAM onchip có dung lượng 32 words, mỗi word là 8 bits.

Các bước thực hiện:

Bước 1. Tạo một project Quartus mới, đặt tên: user_dir/lab5/lab5_part1

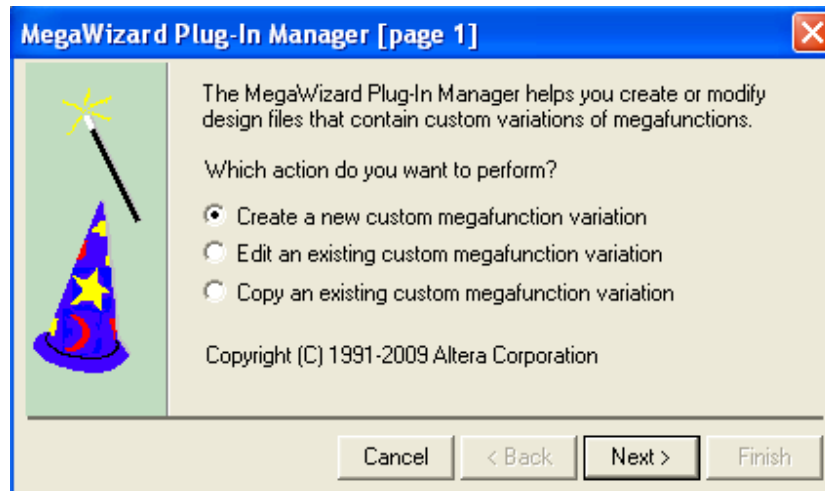
Bước 2. Sử dụng công cụ MegaWizard Plug-in Manager trên Quartus để tạo ra RAM như yêu cầu.

➤ **Tools ➔ MegaWizard Plug-in Manager**



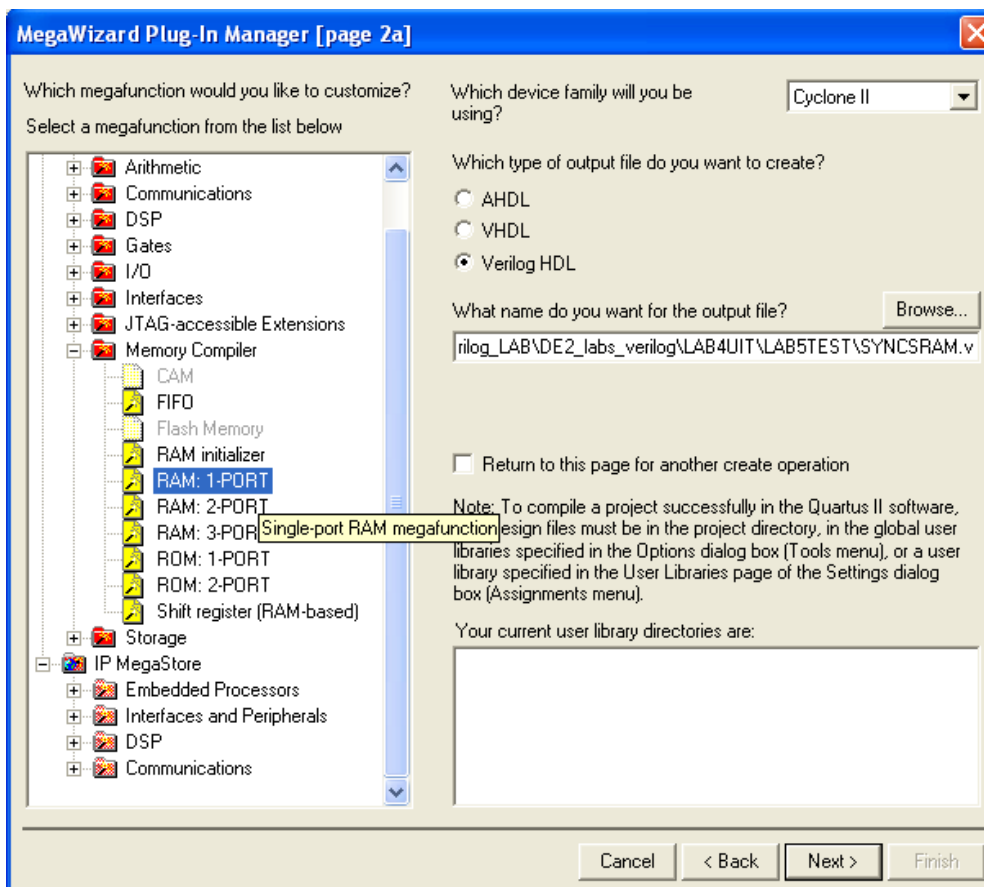
Hình 1.1 Dùng Tool tạo Mega Wizard

➤ **Chọn option “Creat a new custom megafunction variation”**



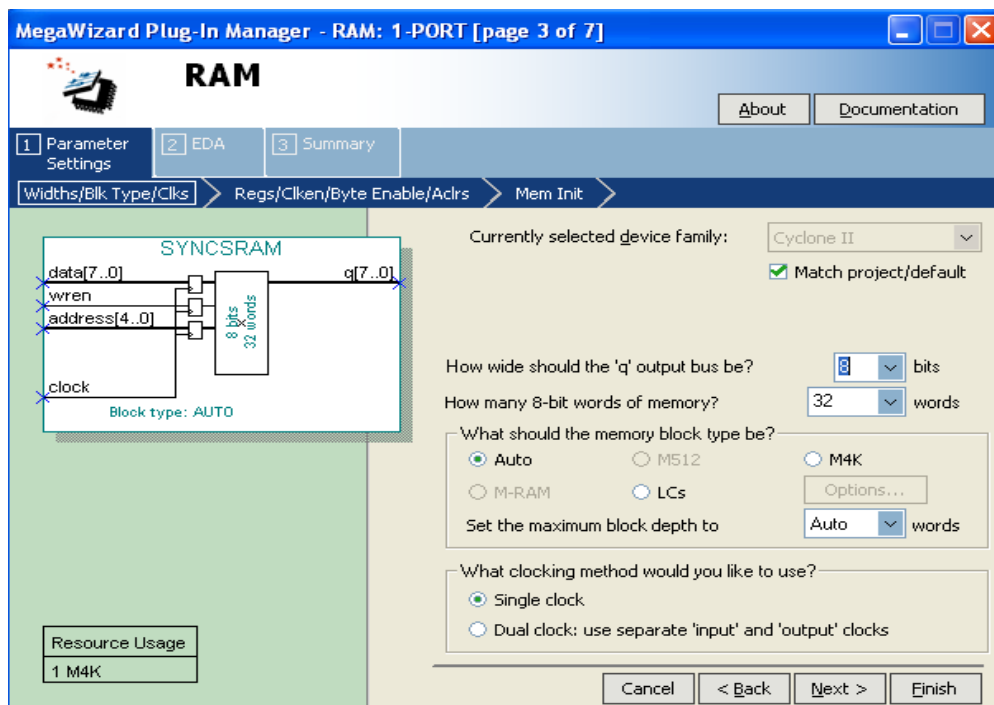
Hình 1.2 Tạo một Mega mới

- Chọn **RAM: 1-PORT** trong danh sách các megafunction
- Chọn device family: **Cyclone II**
- Chọn loại dữ liệu ngõ ra: **Verilog HDL**
- Chọn đường dẫn và tên cho file output



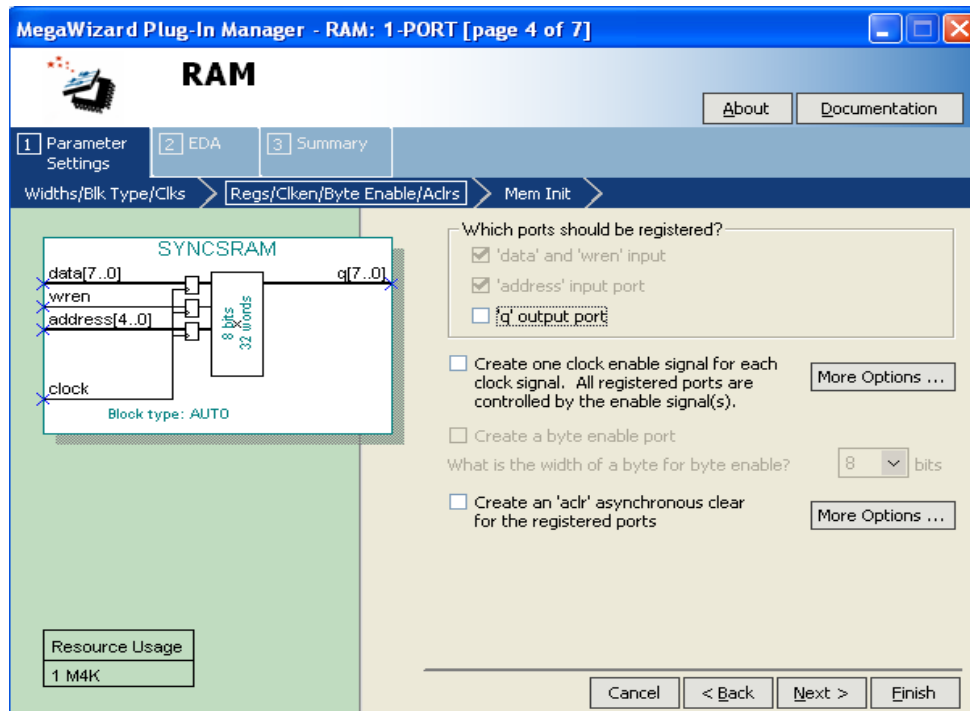
Hình 1.3 Chọn các thông số như hình vẽ

- Nhấn **Next**
- Nhập các thông số như yêu cầu thiết kế.



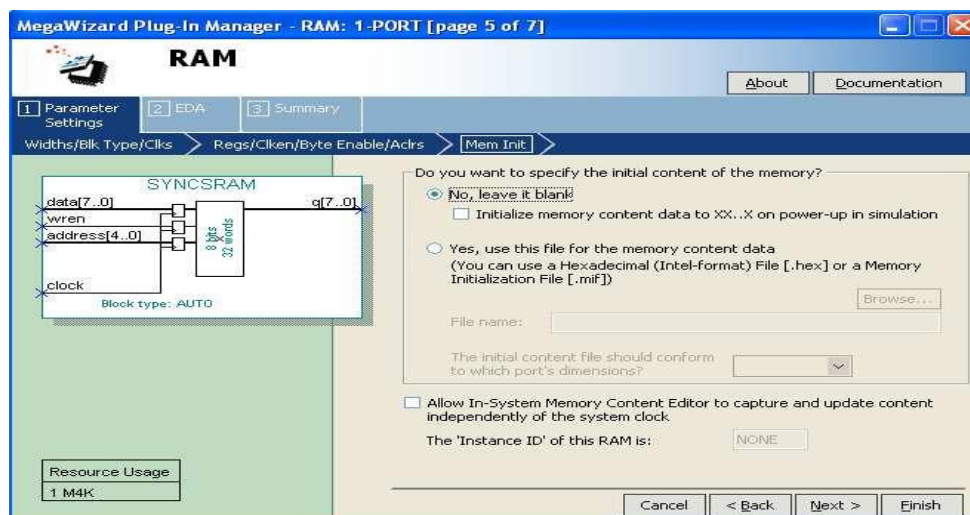
Hình 1.4 Chọn thông số như trên hình

- Nhấn **Next**
- Chọn các option như hình dưới



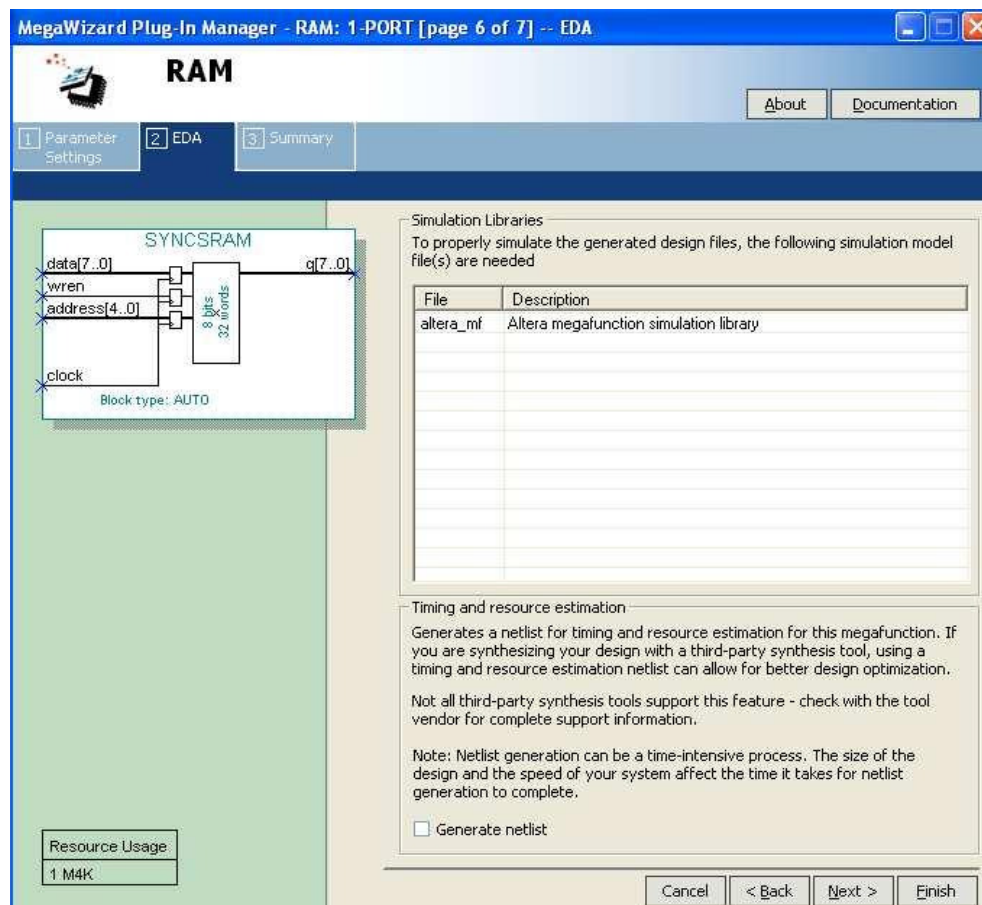
Hình 1.5 Gán registers cho inputs

- Nhấn Next
- Chọn option “No, leave it blank”, bởi vì ta không muốn bộ nhớ có dữ liệu sẵn.



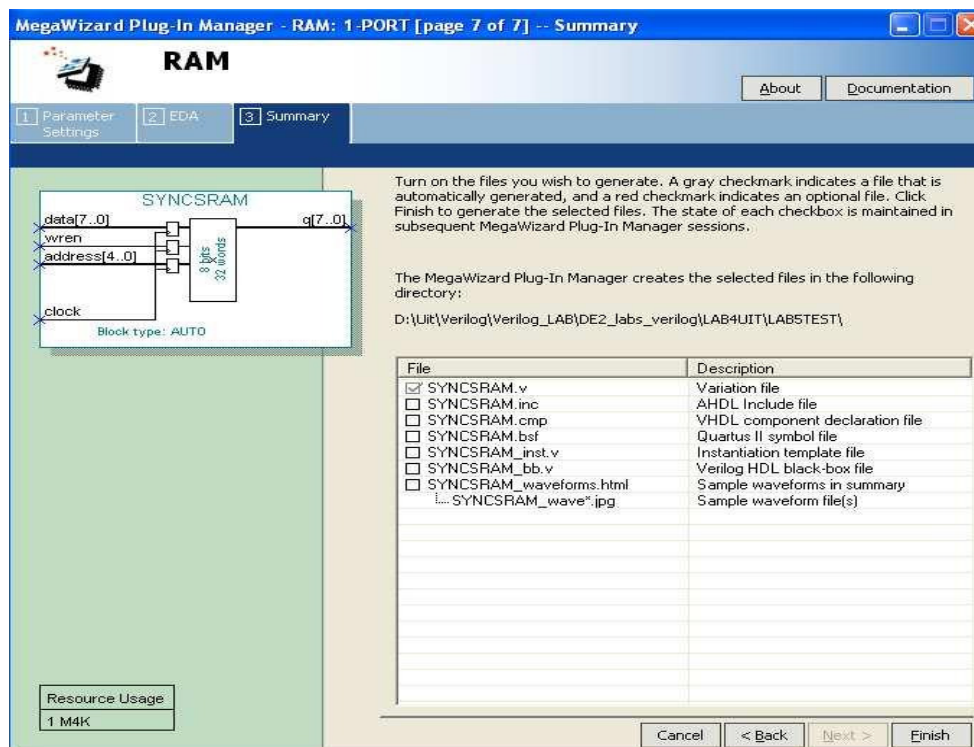
Hình 1.6 Không tạo giá trị ban đầu cho SRAM

- Nhấn Next



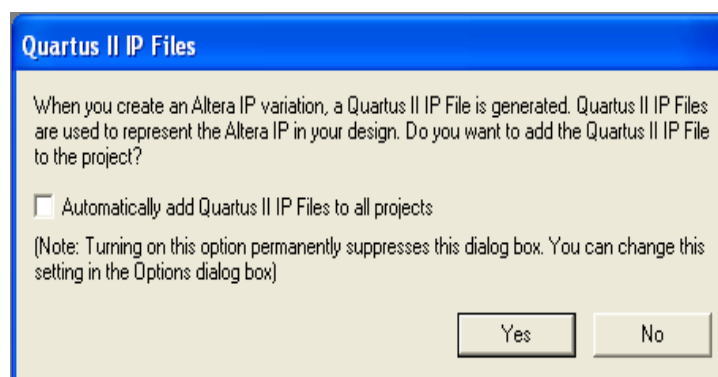
Hình 1.7 Simulation library

- Nhấn **Next**
- Chọn option tạo ra file mô tả thiết kế.v



Hình 1.8 Chọn các loại dữ liệu cần tạo ra

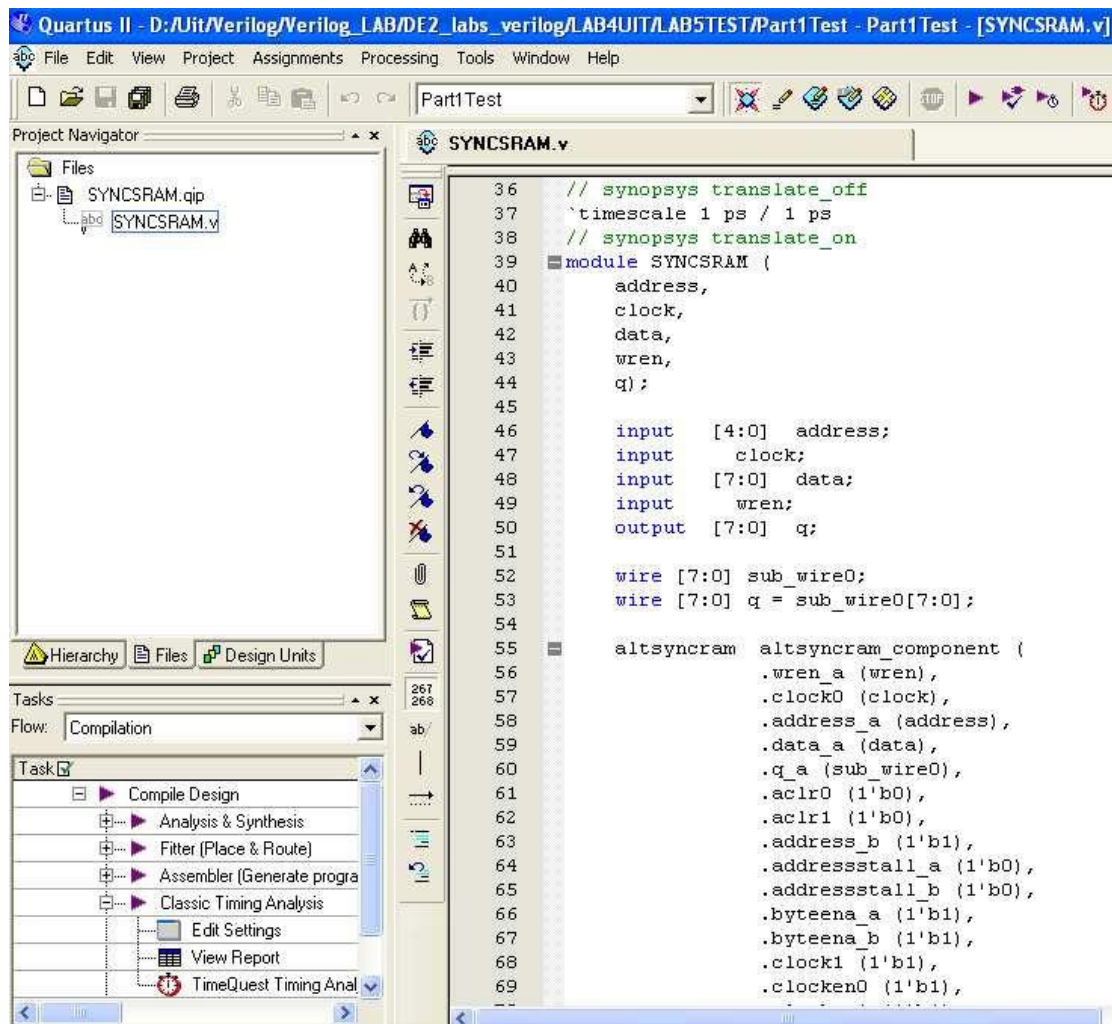
➤ Nhấn **Next**



Hình 1.9 Add dữ liệu tạo ra vào project hiện hành

➤ Nhấn **Yes**

- Một mô tả thiết kế verilog cho một bộ nhớ SRAM được tạo ra.



Hình 1.10 Mô tả verilog của SRAM vừa tạo ra

Bước 3. Biên dịch để phân tích, tổng hợp và tạo ra file .sof.

Bước 4. Tạo file Vector Waveform (.vwf) và chạy mô phỏng để kiểm tra hoạt động của mạch (ghi dữ liệu vào SRAM, sau đó đọc các dữ liệu đã ghi xem SRAM có hoạt động đúng không).

Chú ý : Sinh viên cần chuẩn bị trước ở nhà những công việc sau (Không có bài chuẩn bị không được vào lớp làm thí nghiệm ➔ Tính vắng buổi đó)

 Đọc và thực hiện các bước từ 1 đến 4 ở nhà.

1.1.1.2 Phần 2

Phần này ta dùng Switches trên kit DE2 để ghi dữ liệu vào SRAM đã tạo ra ở phần 1. Sau đó dùng Leds 7 đoạn để hiển thị dữ liệu được đọc ra từ SRAM

Các bước thực hiện

Bước 1. Tạo một project Quartus mới, đặt tên: user_dir/lab5/lab5_part2

Bước 2. Add file mô tả thiết kế đã tạo ra trong phần 1 vào project. Tạo một top-module để instantiate module SRAM.

Bước 3. Gán pin như sau

- Dùng switches SW[7:0] làm tín hiệu dữ liệu ngõ vào.
- Dùng switches SW[15:11] làm tín hiệu địa chỉ.
- Dùng HEX0, HEX1 hiển thị tín hiệu dữ liệu ngõ vào.
- Dùng HEX2, HEX3 hiển thị tín hiệu địa chỉ.
- Dùng HEX4, HEX5 hiển thị tín hiệu ngõ ra.
- Dùng SW[17] làm tín hiệu WREN (khi WREN = 1, cho phép ghi, WREN = 0, cho phép đọc)
- Dùng KEY0 làm xung CLK.

Bước 4. Biên dịch để phân tích, tổng hợp và tạo ra file .sof.

Bước 5. Tạo file Vector Waveform (.vwf) và chạy mô phỏng để kiểm tra hoạt động của mạch.

Bước 6. Nạp file thực thi .sof lên FPGA. Kiểm tra hoạt động của mạch.

Chú ý : Sinh viên cần chuẩn bị trước ở nhà những công việc sau (Không có bài chuẩn bị không được vào lớp làm thí nghiệm ➔ Tính vắng buổi đó)

 Đọc và thực hiện các bước từ 1 đến 5 ở nhà.

1.1.1.3 Phần 3

Thay vì sử dụng các module thiết kế có sẵn trong LPM do Altera cung cấp, ta cũng có thể tự mô tả thiết kế bởi riêng mình sử dụng ngôn ngữ Verilog. Trong phần này ta sẽ tự thiết kế một bộ nhớ SRAM có dung lượng 32 words, mỗi word 8 bits như phần 1 mà không sử dụng module có sẵn. Để dễ hiểu, ta có thể xem bộ nhớ SRAM trên như là 32 thanh ghi, mỗi thanh ghi có độ dài 8 bits. Hoạt động đọc và ghi của các thanh ghi này được điều khiển bởi tín hiệu WREN. Mỗi thanh ghi được gán một địa chỉ xác định.

Các bước thực hiện:

Bước 1. Tạo một project Quartus mới

user_dir/lab5/lab5_part3

Bước 2. Sử dụng Verilog HDL mô tả thiết kế bộ nhớ SRAM như ở trên.

Bước 3. Gán pin như sau:

- Dùng switches SW[7:0] làm tín hiệu dữ liệu ngõ vào.
- Dùng switches SW[15:11] làm tín hiệu địa chỉ.
- Dùng HEX0, HEX1 hiển thị tín hiệu dữ liệu ngõ vào.
- Dùng HEX2, HEX3 hiển thị tín hiệu địa chỉ.
- Dùng HEX4, HEX5 hiển thị tín hiệu ngõ ra.
- Dùng SW[17] làm tín hiệu WREN (khi WREN = 1, cho phép ghi, WREN = 0, cho phép đọc)
- Dùng KEY0 làm xung CLK.

Bước 4. Biên dịch để phân tích, tổng hợp và tạo ra file .sof.

Bước 5. *Thử viết một testbench đơn giản và chạy mô phỏng dùng ModelSim để kiểm tra hoạt động và timing của SRAM. (Hướng dẫn sử dụng ModelSim được trình bày trong phần đính kèm).*

Bước 6. Nạp file thực thi .sof lên FPGA. Kiểm tra hoạt động của mạch.

Chú ý : Sinh viên cần chuẩn bị trước ở nhà những công việc sau (Không có bài chuẩn bị không được vào lớp làm thí nghiệm -> Tính vắng buổi đó)

Đọc và thực hiện các bước từ 1 đến 5.

1.1.1.4 Phần 4

Trên Kit DE2 có một con chip SRAM bất đồng bộ (offchip memory) có dung lượng 256 K 16-bit words (512K 8-bit words). Các pin giao tiếp SRAM gồm:

- Bus ADDRESS[17:0]
- Bus DATA [15:0]
- Chip Enable (/CE): tích cực mức thấp.
- Write Enable (/WE): mức thấp cho phép ghi, khi đọc tín hiệu này phải ở mức cao.
- Output Enable (/OE): mức thấp thì tín hiệu ngõ ra mới valid.
- Upper Byte (/UB) và Lower Byte (/LB). Nếu cả UB và LB đều tích cực mức thấp thì 16 bits ngõ ra D[15:0] khi đọc đều valid. Nếu chỉ UB tích cực mức thấp thì chỉ 8 bits cao D[15:8] khi đọc ra sẽ valid, ngược lại nếu chỉ LB tích cực mức thấp thì chỉ 8 bits thấp D[7:0] khi đọc ra sẽ valid.

Phần này ta sẽ tìm hiểu cách thức sử dụng offchip SRAM.

Các bước thực hiện:

Bước 1. Tạo một project Quartus mới, đặt tên:

user_dir/lab5/lab5_part4

Bước 2. Sử dụng Verilog HDL mô tả thiết kế một module giao tiếp giữa offchip SRAM và các SW, KEY, LEDs.

Bước 3. Gán pin như sau

- Dùng switches SW[7:0] làm tín hiệu dữ liệu ngõ vào.
- Dùng switches SW[15:8] làm tín hiệu địa chỉ (với 8 địa chỉ này sẽ giải mã ra một phần dung lượng của SRAM, ADDR[17:0] = {10'h000, SW[15:8]}).
- Dùng HEX0-HEX3 hiển thị tín hiệu ngõ ra.
- Dùng KEY[0] làm tín hiệu /OE.
- Dùng KEY[1] làm tín hiệu /WE

➤ Nối mức logic “0” cho cả/LB và /UB.

Bước 4. Biên dịch để phân tích, tổng hợp và tạo ra file .sof.

Bước 5. Tạo file Vector Waveform (.vwf) và chạy mô phỏng để kiểm tra hoạt động của mạch.

Bước 6. Nạp file thực thi .sof lên FPGA. Kiểm tra hoạt động của mạch.

Chú ý : Sinh viên cần chuẩn bị trước ở nhà: Đọc và thực hiện các bước từ 1 đến 5

END